

NetSentinel-XDR

Port Scanning: Code & Process Explained

A study guide: button → function → API → function → file → function → actual scanner → result → dashboard

Important note before you read on: the current dashboard has a single button, *"Scan Network"*. It does not have a text box for typing one target IP. So the button always triggers a **discovery scan** of your whole local network (CIDR range), and port scanning happens as part of that: once devices are found, each device's ports are checked. The codebase also contains a separate `scan_target()` function built for scanning one single IP, and the backend API (`/api/scan`) already accepts a target field for it — but no button in the current HTML sends that field yet, so that path is reachable only if someone calls the API directly. This guide explains the real, button-triggered path in full, and covers the single-target path briefly at the end so you can mention it if asked.

1. The Simple Flow (Real Function Names)

```
User
|
"Scan Network" button (id="btn-scan-network") [index.html]
|
startScan() [frontend JS]
|
POST /api/scan { } [HTTP request]
|
api_scan() [app.py]
|
start_scan("discovery") [app.py]
|
discovery_scan_job(job_id) [app.py, runs in a background thread]
|
scanner.get_local_network_info() [scanner.py] -> finds interface + CIDR
|
scanner.discover_devices(cidr) [scanner.py] -> ARP sweep (scapy) finds live IPs
|
scanner.run_nmap_multi_host(ips) [scanner.py] -> PRIMARY port scan (nmap)
|   (if nmap result unusable for a host)
scanner.fill_fallback_port_data() [scanner.py] -> FALLBACK port scan
|
scanner.quick_port_scan(ip, ports) [scanner.py]
|
scanner.check_port(ip, port) [scanner.py] -> socket.connect_ex()
|
Result: device dict with open_ports + services
|
update_device_info() stores it / job["results"] = results [app.py]
|
Frontend polls: GET /api/scan/<job_id> via pollScanJob() [frontend JS]
|
When status == "completed": refreshDevices() -> GET /api/devices -> api_devices() [app.py]
|
renderDevices() [frontend JS]
|
Dashboard table, "Open Ports" column (formatPorts())
```

API (a way for frontend and backend to communicate). Endpoint (a specific API address, e.g. /api/scan). Socket (a software connection used for network communication). Fallback (an alternative method used automatically when the main method fails or gives no result). Thread (a separate execution path that lets the scan run in the background without freezing the page). CIDR (notation such as 192.168.1.0/24 that describes a whole range of IP addresses).

2. Button → Frontend

The scanning feature starts from one HTML button in the dashboard:

```
<button id="btn-scan-network" type="button" class="btn btn-primary">Scan Network</button>
```

This button is wired to a JavaScript function using an event listener:

```
el.btnScanNetwork.addEventListener("click", startScan);
```

Function: startScan()

```
function startScan() {
  if (scanRequestInFlight) return;
  scanRequestInFlight = true;
  el.btnScanNetwork.disabled = true;
  el.scanStatusText.textContent = "Starting network scan...";

  apiPostJson("/api/scan", {})
    .then(function (data) {
      el.scanStatusText.textContent = "Scan running...";
      jobPollAttempts = 0;
      pollScanJob(data.job_id);
    })
    .catch(function (err) { ... });
}
```

```
}
```

What it does (simple English):

1. Disables the button so the user cannot click it twice while a scan is running.
2. Sends an empty JSON body {} to the backend with a POST request to /api/scan — no target IP is included, because there is no input box for one.
3. The backend replies with a job_id (a scan is answered instantly with just an ID; the real scanning happens afterwards in the background).
4. It hands that job_id to pollScanJob(), which checks the status every 1.5 seconds until the scan is finished.

Next function it calls: pollScanJob(jobId) → repeatedly calls GET /api/scan/<jobId> until job.status is no longer "running".

3. Frontend → API → app.py

```
startScan() [frontend]
|
POST /api/scan          (HTTP method: POST, endpoint: /api/scan)
|
app.py
|
api_scan() [Flask route function]
```

Function: api_scan()

```
@app.route("/api/scan", methods=["POST"])
def api_scan():
    payload = request.get_json(silent=True) or {}
    target = payload.get("target")

    validated_target = None
    if target is not None:
        validated = is_valid_ip_or_network(target)
        if validated is False:
            return jsonify({"error": "invalid target IP or CIDR"}), 400
        validated_target = validated

    if not scan_lock.acquire(blocking=False):
        return jsonify({"error": "a scan is already in progress"}), 409

    try:
        if validated_target is not None:
            job_id = start_scan("target_scan", validated_target)
        else:
            job_id = start_scan("discovery")
    except Exception:
        scan_lock.release()
        return jsonify({"error": "failed to start scan"}), 500

    return jsonify({"job_id": job_id, "status": "running", ...}), 202
```

What app.py receives: an empty payload {} from the current dashboard button (no target key), so target is None.

What it checks/validates: it only validates a target IP if one was sent. Since none is sent, validation is skipped and validated_target stays None. It also checks scan_lock (a lock is a flag that blocks two scans from running at the same time) so only one scan runs at once.

Which function it calls next: because validated_target is None, it calls start_scan("discovery"), not the target-scan path.

Response sent back: a JSON object containing the new job_id, so the frontend can poll for the result.

4. app.py → scanner.py (Following Every Call)

```
api_scan()
|
start_scan("discovery")
|
discovery_scan_job(job_id)    <-- runs in its own background Thread
|
scanner.get_local_network_info()
|
scanner.discover_devices(cidr)
|
scanner.run_nmap_multi_host(ips) --> primary port-scan mechanism
```

```
|
scanner.fill_fallback_port_data() --> fallback port-scan mechanism (only if needed)
```

Function: start_scan() **File:** app.py

Called by: api_scan()

Calls next: discovery_scan_job() (started as a new background Thread)

Purpose: Creates a job record (id, status "running") in the shared scan_jobs dictionary, then starts the actual scanning work on a separate thread so the API can reply immediately.

```
def start_scan(job_type, target=None):
    job_id = str(uuid.uuid4())
    scan_jobs[job_id] = {"id": job_id, "status": "running", ...}
    if job_type == "discovery":
        thread = threading.Thread(target=discovery_scan_job, args=(job_id,), daemon=True)
    else:
        thread = threading.Thread(target=target_scan_job, args=(job_id, target), daemon=True)
    thread.start()
    return job_id
```

Explanation:

threading.Thread(...) → Thread (a separate execution path) so scanning does not freeze the API response.

thread.start() → begins running discovery_scan_job() in the background.

return job_id → sent straight back to the frontend so it can poll for progress.

Function: discovery_scan_job() **File:** app.py

Called by: start_scan() (as the thread's target function)

Calls next: scanner.get_local_network_info() then scanner.discover_devices()

Purpose: This is the background worker. It figures out the local network range, asks scanner.py to scan it, and then saves whatever comes back into the job record and the shared devices list.

```
def discovery_scan_job(job_id):
    interface, cidr = scanner.get_local_network_info()
    results = scanner.discover_devices(cidr)
    for found_device in results:
        update_device_info(found_device)
    job = scan_jobs.get(job_id)
    job["status"] = "completed"
    job["results"] = results
```

Explanation:

scanner.get_local_network_info() → finds which network interface (e.g. eth0) and CIDR (IP range) the machine is on.

scanner.discover_devices(cidr) → the real work: finds live devices AND scans their ports (explained fully in Section 5).

update_device_info(found_device) → saves each device's data (including open ports) into the app's shared devices dictionary.

job["results"] = results → stores the full result so the frontend can read it later.

5. The Actual Port-Scanning Mechanism

Port scanning happens inside scanner.discover_devices(cidr), in two stages: first it finds which IPs are alive (this part uses **scapy** ARP requests, not port scanning), then for those live IPs it checks which ports are open. There are two mechanisms for the port-checking part:

Primary mechanism: Nmap (used first, if the nmap program is installed)

Function: run_nmap_multi_host() **File:** scanner.py

Called by: discover_devices()

Calls next: run_command() to launch nmap, then parse_nmap_output_multi() to read the result

Purpose: Runs one nmap command against ALL discovered IPs at once, asking it to scan ports, guess the service running on each open port (-sV), and guess the operating system (-O).

```
def run_nmap_multi_host(ips, timeout=None):
    args = [
        "nmap", "-Pn", "-sV", "-O", "-T4",
        "--host-timeout", NMAP_MULTI_HOST_TIMEOUT,
        "--max-retries", "1",
        "-oX", "-",
    ] + ips
    proc = run_command(args, timeout=timeout)
    if not proc or not proc.stdout:
        return {}
    return parse_nmap_output_multi(proc.stdout)
```

Explanation:

"nmap", "-Pn", "-sV", "-O", "-T4" → nmap (a network scanning program) is run as a command-line process; -Pn skips a "ping first" check, -sV detects service versions, -O guesses the OS, -T4 is a faster scan speed.

"-oX", "-" → tells nmap to print its result as XML text instead of a file.

run_command(args, ...) → actually launches nmap as a subprocess and captures its output.

parse_nmap_output_multi(proc.stdout) → converts the raw XML text into a Python dictionary Claude/app.py can use.

Fallback mechanism: Python socket (used only when nmap is missing, or gives no usable result for a particular host — decided by nmap_result_is_usable())

Function: quick_port_scan() **File:** scanner.py

Called by: fill_fallback_port_data()

Calls next: check_port() for every port in the list

Purpose: Goes through a fixed list of common ports one by one and asks check_port() whether each one is open.

```
def quick_port_scan(ip, ports):
    open_ports = []
    services = []
    for port in ports:
        if check_port(ip, port):
            open_ports.append(port)
            try:
                name = socket.getservbyport(port)
            except OSError:
                name = None
            services.append({"port": port, "protocol": "tcp", "name": name, "product": None})
    return open_ports, services
```

Explanation:

for port in ports → loops over a fixed list of common ports (22, 80, 443, 3389, ...).

check_port(ip, port) → the real port-check, shown below.

socket.getservbyport(port) → looks up the common name of a port (e.g. 80 = "http").

Function: check_port() **File:** scanner.py

Called by: quick_port_scan()

Calls next: nothing (this is the lowest-level check)

Purpose: This is the single most important function: it is the one line of code that actually tests whether one port on one IP is open, using a raw TCP socket instead of nmap.

```
def check_port(ip, port, timeout=0.5):
    try:
        with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
            sock.settimeout(timeout)
            return sock.connect_ex((ip, port)) == 0
    except OSError:
        return False
```

Explanation:

socket.socket(AF_INET, SOCK_STREAM) → Socket (a software connection point) — creates a TCP socket, the same kind of connection a web browser uses.

sock.settimeout(timeout) → gives up after 0.5 seconds if there is no reply, so a dead port does not freeze the scan.

sock.connect_ex((ip, port)) → tries to connect to that exact IP and port.

== 0 → connect_ex returns 0 only when the connection succeeded, meaning the port accepted the connection (it is open).

6. Following the Chain Inside discover_devices()

```
discover_devices(cidr)
|
srp(arp_request, ...) [scapy] --> finds which IPs are alive (not port scanning)
|
shutil.which("nmap") --> checks if nmap is installed on this machine
|
run_nmap_multi_host(ips) --> PRIMARY: nmap scans ports for all IPs at once
|
parse_nmap_output_multi(xml_text)
|
parse_nmap_host_element(host) --> turns nmap's XML into a plain dict per host
|
nmap_result_is_usable(scanned) --> per host: did nmap give a good answer?
|                                     (yes -> keep nmap's ports) (no -> fallback below)
fill_fallback_port_data(devices, fallback_ips) --> quick_port_scan() --> check_port()
```

Why each link exists: nmap is preferred (it also reports service names and OS guesses), but it might be missing, time out, or return nothing useful for one host. So nmap_result_is_usable() is checked per host, and only the hosts that still need it go through the socket-based fallback.

7. Imports Used for Port Scanning

Import	Meaning	Used here for
socket	Python's built-in library for network connections.	check_port() opens a TCP socket and tests whether a port accepts a connection.
subprocess (via run_command)	Lets Python launch an external program and read its output.	run_nmap_multi_host() launches the real "nmap" command-line program.
shutil	File/utility helper library.	shutil.which("nmap") checks whether nmap is actually installed before using it.
xml.etree.ElementTree (as ET)	Parses XML text into a tree of elements.	parse_nmap_output_multi()/parse_nmap_host_element() read nmap's XML output.
concurrent.futures (ThreadPoolExecutor)	Runs several functions in parallel.	fill_fallback_port_data() checks several hosts' ports in parallel instead of sequentially.
scapy (ARP, Ether, srp)	Library for building/sending raw network packets.	Used earlier in discover_devices() to find which IPs are alive on the network.

8. Result Flow — From Scan Back to the Dashboard

```
discover_devices() returns a list of device dicts, e.g.:
{
    "ip": "192.168.1.20",
```

```

    "mac": "aa:bb:cc:dd:ee:ff",
    "hostname": "some-device",
    "online": true,
    "open_ports": [22, 80],
    "services": [{"port": 22, "protocol": "tcp", "name": "ssh", "product": None}, ...],
    "os": "Linux 5.x"
}
|
discovery_scan_job() -> update_device_info(found_device) [app.py]
                        saves it into the shared `devices` dict
|
job["results"] = results ; job["status"] = "completed" [app.py, in scan_jobs]
|
Frontend: pollScanJob(jobId) -> GET /api/scan/<jobId> sees status == "completed"
|
refreshDevices() -> GET /api/devices -> api_devices() [app.py] returns devices list
|
renderDevices(devices) [frontend JS]
|
formatPorts(device.open_ports) -> "Open Ports" column on the dashboard table

```

Function: formatPorts() — the very last step, turning the raw list of port numbers into readable text for the table:

```

function formatPorts(ports) {
  if (!Array.isArray(ports) || ports.length === 0) return "Not detected";
  return ports.join(", ");
}

```

9. The Alternate Single-Target Path (scan_target)

The codebase also contains a ready-made path for scanning just ONE IP address: if the frontend ever sent `{"target": "192.168.1.20"}` instead of an empty body, `api_scan()` would call `start_scan("target_scan", ip) → target_scan_job() → scanner.scan_target(target) → scanner.run_nmap(ip)` (single-host nmap), falling back to `scanner.fallback_scan(ip) → quick_port_scan() → check_port()` if nmap is unusable. No button in the current HTML sends that target field yet, so this path exists in the code but is not currently reachable from the dashboard.

10. Simple Walkthrough Example

You click "Scan Network". Suppose your network is 192.168.1.0/24 and one live device found is 192.168.1.20, with ports 22, 80, and 443 open.

1. `startScan()` sends POST `/api/scan` with an empty body.
2. `api_scan()` sees no target, calls `start_scan("discovery")`.
3. `discovery_scan_job()` gets the CIDR 192.168.1.0/24 from `get_local_network_info()`.
4. `discover_devices("192.168.1.0/24")` ARP-sweeps the range and finds 192.168.1.20 is alive.
5. Because nmap is installed, `run_nmap_multi_host(["192.168.1.20"])` scans it and reports ports 22 (ssh), 80 (http), 443 (https) as open.
6. `nmap_result_is_usable()` says yes, so the fallback socket scan is skipped for this host.
7. The device dict `{"ip": "192.168.1.20", "open_ports": [22, 80, 443], ...}` is saved via `update_device_info()` and placed in `job["results"]`.
8. The frontend keeps polling, sees status: "completed", then calls `refreshDevices()`.
9. The dashboard table shows a row for 192.168.1.20 with "Open Ports: 22, 80, 443".

11. Likely Viva Questions

Q: Which button starts port scanning?

Short Answer: The "Scan Network" button (id="btn-scan-network").

Simple Explanation: It calls `startScan()`, which POSTs to `/api/scan` with no target, so it always runs a full discovery scan.

Q: Which Flask function receives the request?

Short Answer: `api_scan()`, for the route POST `/api/scan`.

Simple Explanation: It reads the JSON body, checks for a target, and decides discovery vs target_scan.

Q: Which file contains the actual port-checking logic?

Short Answer: scanner.py.

Simple Explanation: Specifically check_port() (socket) and run_nmap_multi_host() (nmap).

Q: Why is nmap used?

Short Answer: Because it reports service names and OS guesses, not just open/closed.

Simple Explanation: run_nmap_multi_host() runs it with -sV (service version) and -O (OS detection).

Q: What happens if nmap is unavailable or fails for a host?

Short Answer: The code falls back to a plain Python socket scan.

Simple Explanation: fill_fallback_port_data() calls quick_port_scan(), which calls check_port() using socket.connect_ex().

Q: How does check_port() decide a port is open?

Short Answer: It tries socket.connect_ex((ip, port)) and checks if the result equals 0.

Simple Explanation: A return value of 0 means the TCP connection succeeded, i.e. the port accepted it.

Q: How does the frontend know the scan is finished?

Short Answer: It polls GET /api/scan/ every 1.5 seconds with pollScanJob().

Simple Explanation: When job.status changes from "running" to "completed" (or "failed"), polling stops.

Q: Where is the scan result stored on the backend?

Short Answer: In the scan_jobs dictionary (job["results"]) and in the shared devices dictionary.

Simple Explanation: discovery_scan_job() writes both after scanner.discover_devices() returns.

Q: How does the dashboard display the result?

Short Answer: renderDevices() builds a table row per device, using formatPorts() for the ports column.

Simple Explanation: formatPorts() just joins the open_ports list into a readable comma-separated string.

Q: Does the current UI support scanning a single specific IP?

Short Answer: The backend can (scan_target()/run_nmap()), but no button currently sends a target IP.

Simple Explanation: The "Scan Network" button always sends an empty body, so it triggers discovery, not a single-target scan.

12. Final 30-Second Explanation (Memorize This)

"Sir, when I click Scan Network, the frontend function startScan() sends a POST request to /api/scan with no target, so the backend function api_scan() starts a discovery scan through start_scan(). This runs discovery_scan_job() in a background thread, which calls scanner.discover_devices(). That function first finds live devices on the network using ARP, then scans their ports: it tries nmap first through run_nmap_multi_host(), and if nmap doesn't give a usable result for a device, it falls back to a plain Python socket check using quick_port_scan() and check_port(), which uses socket.connect_ex() to test each port directly. The results, including open ports and services, are saved into the device list and the scan job. The frontend keeps polling /api/scan/<job_id> until it's done, then calls refreshDevices() to pull the updated device list from /api/devices, and renderDevices() displays the open ports on the dashboard table."